Arabic Sentiment Analysis with AraBERT

An illustrated explanation of the data, model, fine-tuning workflow and evaluation protocol.



Figure 1. AI-generated conceptual illustration. The network layers are illustrative, not an exact model diagram.

2	42	3
sentiment classes	notebook cells	full-mode epochs

Project repository

github.com/yassinelaouni/arabert-sentiment-analysis

23 OCTOBER 2026 / ENGLISH PROJECT EXPLAINER

01 / THE IDEA

How AraBERT learns sentiment

A pretrained language model becomes a classifier when it learns from labeled examples. The notebook explains this process in French. [1, 4]

Pretraining

AraBERT first learns useful Arabic language patterns from large text collections, including through masked-word prediction. This creates reusable language representations. [4]

Fine-tuning

The project adds a two-class classification head and trains it with labeled tweets. The encoder weights are updated too, so the whole network adapts to sentiment. [1]

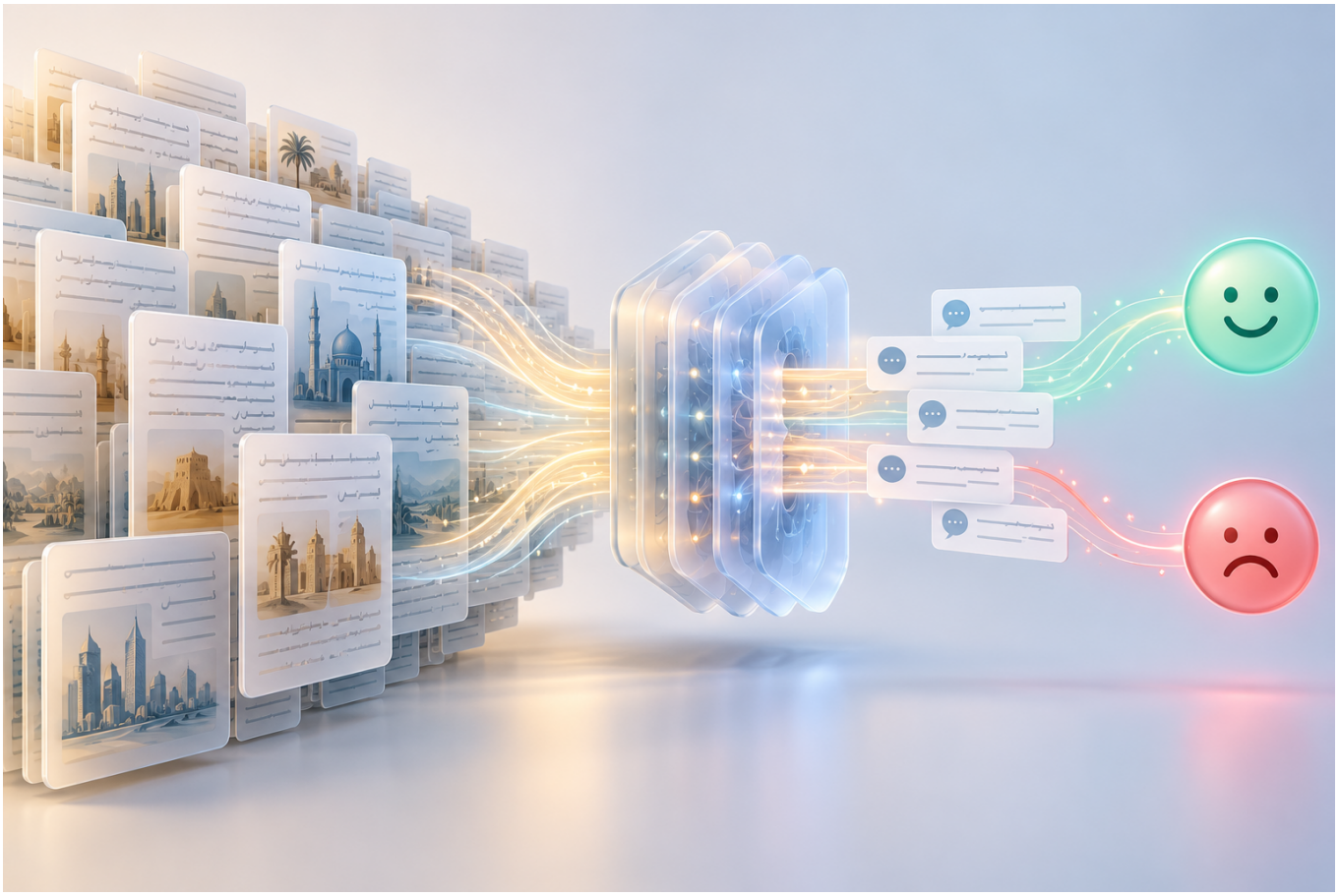


Figure 2. AI-generated concept: language knowledge is reused for a focused sentiment task. Symbols and layers are illustrative.

Selected model: aubmindlab/bert-base-arabertv02. This is the unsegmented v0.2 variant; Farasa segmentation is disabled. [4, 5]

02 / THE DATA

Start with the real corpus

The Kaggle archive was downloaded and its four TSV files were inspected locally. The official training and test partitions are retained. [1, 2]

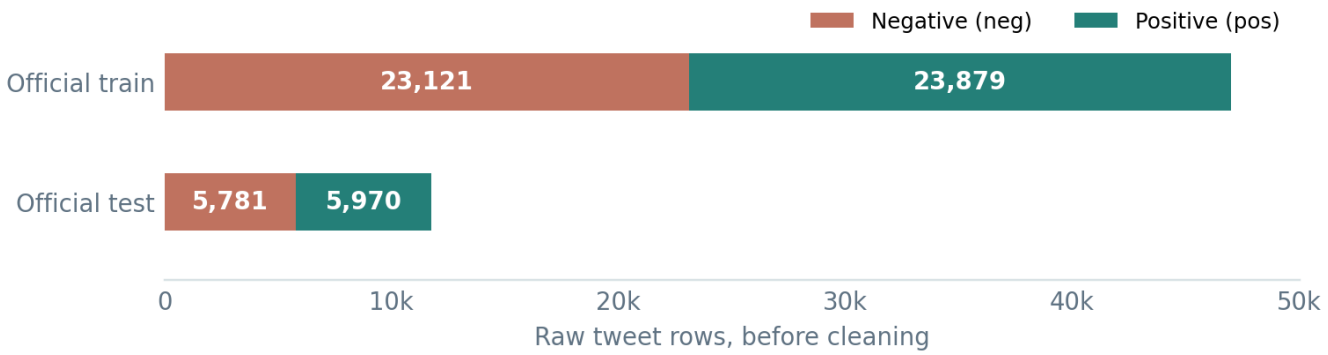
58,751
raw tweet rows

47,000
official train rows

11,751
official test rows

Original file	Label	Rows
train_Arabic_tweets_negative_20190413.tsv	neg	23,121
train_Arabic_tweets_positive_20190413.tsv	pos	23,879
test_Arabic_tweets_negative_20190413.tsv	neg	5,781
test_Arabic_tweets_positive_20190413.tsv	pos	5,970

Format: UTF-8; tab-separated; no header; two columns: label, then tweet. **Mapping:** neg = 0 = negative; pos = 1 = positive. There is no neutral class.



Measured class counts from the original files, before removing duplicates or contradictory labels.

The archive also contains individual text files representing the corpus. Loading those alongside the TSVs would duplicate data. The corpus author describes emoji-based collection and labeling, a source of annotation noise. [3]

03 / DATA PREPARATION

Protect the evaluation

Duplicate tweets can make a model appear better than it is. The notebook checks both normalized text and the exact token sequences given to the model. [1]

1. **Validate the input**
   Inspect real files, labels and encoding. Remove missing or empty texts, including texts that become empty after preprocessing.

2. **Use the official preprocessor**
   Preserve negation and emojis, keep expressive repetitions, and avoid extra stopword removal or stemming. Farasa is not used for v0.2.

3. **Give the official test priority**
   Remove overlapping training text using test inputs only. Split the cleaned development pool into stratified train and validation partitions.

4. **Resolve duplicate groups**
   Within each partition, remove all examples in a conflicting-label group. Retain one copy of a consistent repeated text.

5. **Check the model input as well**
   Compare token sequences after truncation and unknown-token handling. Remove overlaps with priority: test, then validation, then train.

14,592

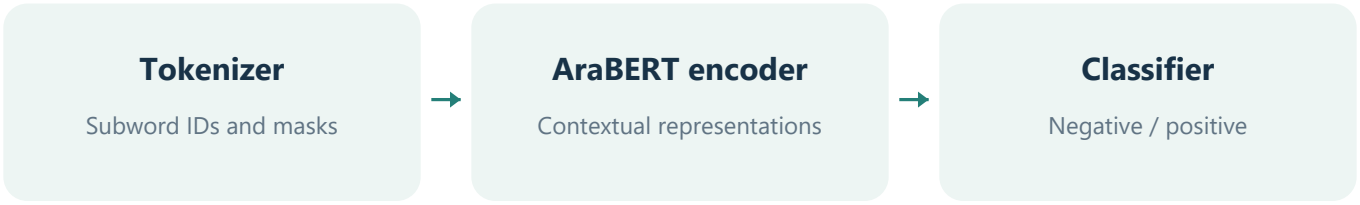
Training rows removed because their normalized text also appeared in the official test. This was measured during local preparation.

This is a data-quality finding, not a model performance score. The reported test set is the official test after documented cleaning; it is never moved into training. Exact-duplicate checks do not detect every paraphrase.

04 / FINE-TUNING

Train the complete network

PyTorch performs the weight updates. Hugging Face Trainer handles batching, validation, checkpoints and selection of the best model. [1, 6]



Simplified data flow. AutoModelForSequenceClassification loads the backbone and adds the classification head; all parameters remain trainable.

Setting	Notebook configuration
Learning rate	2e-5 (0.00002)
Epochs	3 full mode; 1 quick mode
GPU batch size	8 below 20 GiB; otherwise 16
Weight decay	0.01
Gradient accumulation	Targets an effective batch of at least 16
Schedule	Linear decay with 10% warmup
Checkpoint selection	Best validation macro F1, each epoch

Length and memory are data-driven

Sequence length is chosen from the 95th percentile of training-token lengths, rounded up to a multiple of 8 and capped at 256 and the model limit. Dynamic padding avoids padding every tweet to the same fixed length.

Hardware-aware execution

Use BF16 or FP16 only when the CUDA hardware supports it and the notebook probe succeeds. CPU and MPS use float32. Gradient checkpointing and accumulation reduce memory pressure. External tracking integrations are disabled.

05 / EVALUATION

Measure before making claims

Use validation to select the model. Evaluate the held-out test only after that selection, then inspect per-class results and individual mistakes. [1]

Accuracy

The fraction of test examples assigned the correct class.

Precision

For a predicted class, the fraction of those predictions that are correct. Macro precision averages the two class values.

Recall

For a real class, the fraction of its examples that the model finds. Macro recall averages both class values.

Macro F1

Compute F1 from precision and recall for each class, then average the two F1 values equally. This selects the checkpoint.

The notebook produces: training/validation loss curves, accuracy, macro precision/recall/F1, per-class F1, a classification report, a confusion matrix and an error table with probabilities.

Verified on CPU

- Inspection and preprocessing of 58,751 real rows.
- Preparation and split checks for 256 / 128 / 128 examples.
- One optimization step using 4 train, 4 validation and 4 test examples.
- Evaluation, export and reload; probabilities agree within 1e-4.

Still to run

- The standard 256-example quick training run.
- The full three-epoch experiment on the cleaned corpus.
- CUDA and mixed-precision execution on a GPU.

No full-corpus accuracy or F1 is claimed. The four-example technical test checks execution, not generalization. Verification evidence was recorded on 15 October 2026; this guide was prepared on 23 October 2026.

06 / USE THE PROJECT

Run it, then predict

Start with the small quick mode, check that every cell succeeds, then restart the kernel and run the complete experiment. Python 3.10-3.12 is required by this dependency stack. [1]

1. Create a local Python environment (Windows)

```
git clone https://github.com/yassinelaouni/arabert-sentiment-analysis.git
cd arabert-sentiment-analysis
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m ipykernel install --user --name arabert
.\.venv\Scripts\python.exe -m jupyterlab arabert_sentiment_analysis.ipynb
```

2. Provide the data

Download the four original TSVs into **data/**, or set **DATA_DIR** in the notebook. On Kaggle, attach the dataset and enable Internet; on Colab, upload/extract it or set a Drive path. Kaggle credentials stay outside the notebook.

3. Execute the notebook in order

Select the **arabert** kernel locally. Keep **QUICK_MODE=True** for the first run. For full training, set **QUICK_MODE=False**, restart the kernel and rerun all cells. If packages were already imported before installation, restart once installation finishes.

4. Use the prediction function after training

```
prediction = predict_sentiment(text)
print(prediction)
```

The result contains a sentiment label and two softmax probabilities. The final notebook cells save the model/tokenizer and reload them with the same preprocessing and length settings.

الخدمة رائعة وأنا سعيد	لا أحب هذه الخدمة
The service is great and I am happy.	I do not like this service.

Demonstration inputs only. No model predictions or probability values are invented for these examples.

07 / PERSPECTIVE

Limits and next steps

A working training pipeline is the starting point. The final model needs independent evaluation across language varieties, topics and annotation conditions.

Sarcasm and context

A short tweet can reverse its apparent meaning through irony or depend on a previous message that the classifier cannot see.

Dialect coverage

Moroccan darija, Arabizi and code-switching may be less well recognized. Add independently labeled examples from the intended audience.

Annotation bias

Emoji-based labels can reward shortcuts. Audit noisy annotations and test robustness with and without emojis. [3]

Uncertain probabilities

Softmax scores are uncalibrated. The two-class model also assigns a polarity to neutral or mixed feelings.

Practical improvements: collect a clean darija test set, search for near-duplicates, compare a tweet-adapted AraBERT on validation, run multiple seeds, calibrate probabilities and add a neutral class only with suitable labeled data.

Sources and project materials

[1] Project repository and verification notes
https://github.com/yassinelaouni/arabert-sentiment-analysis

[2] Kaggle: Arabic Sentiment Twitter Corpus
https://www.kaggle.com/datasets/mksaad/arabic-sentiment-twitter-corpus

[3] Corpus author: collection and annotation
https://github.com/motazsaad/arabic-sentiment-analysis

[4] AraBERT v0.2 model card
https://huggingface.co/aubmindlab/bert-base-arabertv02

[5] Official AraBERT preprocessing implementation
https://github.com/aub-mind/arabert/blob/master/preprocess.py

[6] Hugging Face Trainer, pinned API v4.51.3
https://huggingface.co/docs/transformers/v4.51.3/en/main_classes/trainer

Figures 1-2: generated with the built-in ChatGPT image-generation tool. They are conceptual illustrations, not measured results or exact BERT architecture diagrams. Prompt text and original images accompany this PDF. Dataset counts and preparation findings come from the actual local files.